Microsoft Tunnel Gateway — Deployment, VPN Configuration & Ongoing Operations

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Microsoft Tunnel Gateway: Deployment, VPN Configuration & Ongoing Operations 5

Scope boundary — read this first . . . 5

What you get . . . 5

What this delivers . . . 6

The deployment model . . . 6

Outcome . . . 6

1. Solution Overview — Microsoft Tunnel Gateway: Deployment, VPN Configuration & Ongoing Operations 6

What this solves . . . 6

How Microsoft Tunnel works (the parts you must understand) . . . 6

Client-platform support — stated honestly . . . 7

Split tunnel vs full tunnel — the routing decision . . . 7

Scope — what this solution covers . . . 8

Out of scope (deferred to the customer or another product) . . . 8

Authentication and connections used . . . 8

The standard deployment model . . . 9

2. Requirements Guide — Microsoft Tunnel Gateway: Deployment, VPN Configuration & Ongoing Operations 9

2.1 Intune licensing for Microsoft Tunnel . . . 9

2.2 A supported Linux host for the Gateway . . . 10

2.3 Container runtime (Podman or Docker) . . . 10

2.4 TLS server certificate for the Gateway . . . 10

2.5 Public FQDN + DNS + network path . . . 10

2.6 Outbound connectivity from the Gateway to Microsoft services . . . 11

2.7 Microsoft Defender for Endpoint app (mobile Tunnel client) . . . 11

2.8 Client authentication method decided . . . 11

2.9 Load balancer for high availability (if deploying HA) . . . 11

2.10 Microsoft Graph PowerShell (for the reporting scripts) . . . 11

2.11 Pilot and broad device groups . . . 12

Checklist . . . 12

Global Requirements — Intune Solutions (category-level) 12

G1. Microsoft Entra ID tenant . . . 12

G2. Microsoft Intune subscription . . . 12

G3. Administrative roles . . . 12

G4. Device enrollment in Intune . . . 13

G5. Pilot security group . . . 13
G6. Supported Windows baseline . . . 13
G7. Network access to Microsoft cloud endpoints . . . 13
3. Implementation Guide — Microsoft Tunnel Gateway: Deployment, VPN Configuration & Ongoing Operations 13
Step 0 — Assess (run the prerequisite check first) . . . 14
Step 1 — Create the device groups . . . 14
Step 2 — Create the Site . . . 14
Step 3a — Create the Server Configuration . . . 14
Step 3b — Assign the Server Configuration to the Site . . . 15
Step 4 — Install the Tunnel Gateway on the Linux host . . . 15
Step 5 — Deploy the Microsoft Defender for Endpoint client app (mobile) . . . 16
Step 6 — Create the per-platform VPN connection profiles . . . 16
Step 7 — Conditional Access (report-only first) . . . 17
Step 8 — Apply on a pilot device and validate . . . 17
Step 9 — Pilot gate . . . 18
Step 10 — Expand to production . . . 18
5. Testing Guide — Microsoft Tunnel Gateway: Deployment, VPN Configuration & Ongoing Operations 18
Test 1 — Gateway server is healthy in Intune . . . 18
Test 2 — TLS certificate valid and matches the FQDN . . . 19
Test 3 — VPN connection profile deployed . . . 19
Test 4 — Client connects to the Tunnel . . . 19
Test 5 — Internal resource reachable through an include route . . . 19
Test 6 — Split-tunnel routing behaves as configured . . . 19
Test 7 — Conditional Access (report-only) evaluates as intended . . . 19
Test 8 — High availability failover (if HA deployed) . . . 19
Test 9 — Fleet + health report . . . 19
Pass/fail gate . . . 20
6. Troubleshooting Guide + Operations Runbook — Microsoft Tunnel Gateway 20
The core scenario — "the client won't connect to the Tunnel" . . . 20
A. Gateway server unhealthy / container down . . . 21
B. TLS certificate problems . . . 21
C. Client app missing or VPN profile not applied . . . 22
D. Networking — FQDN, port, or internal routing . . . 22
E. Authentication failures . . . 22
F. Conditional Access blocking . . . 23

G. Diagnostics quick reference . . . 23
Operations Runbook (ongoing support) 23
Daily . . . 23
Weekly . . . 23
Monthly / as needed . . . 24
When to escalate . . . 24
7. Best Practices — Microsoft Tunnel Gateway: Deployment, VPN Configuration & Ongoing Operations 24
Keep-it-clean guidance . . . 24
Where to go next (adjacent catalog areas) . . . 25
8. Deliverables Summary — Microsoft Tunnel Gateway: Deployment, VPN Configuration & Ongoing Operations 26
Documentation . . . 26
Scripts . . . 26
What the buyer receives . . . 27
Outcome . . . 27
Appendix A — PowerShell Scripts 28

Executive Summary — Microsoft Tunnel Gateway: Deployment, VPN Configuration & Ongoing Operations

Intune Solutions

Microsoft Tunnel gives Intune-managed devices a VPN path to on-premises and internal resources — but the Gateway that terminates those connections is a **Linux container service** that has to be installed, TLS-secured, configured, scaled, and kept healthy over time. Most teams can stand up a single test server; far fewer get to a **production** gateway that is highly available, monitored, and supportable a year later. This package takes you the whole distance.

This is a complete, **deploy-it-today** solution written for a competent administrator with no prior Microsoft Tunnel experience. It installs the Tunnel Gateway end to end (Linux host + container runtime + the Microsoft Tunnel agent), secures it with a TLS server certificate, configures the **Server Configuration and Site** in Intune (listener port, DNS, split-tunnel vs full-tunnel routes, client address pool), delivers the **per-platform VPN connection profiles** through Intune, sets up **high availability and scale-out**, and hands over a **monitoring + operations & troubleshooting runbook** for ongoing support.

Scope boundary — read this first

This solution is the **device-tunnel Tunnel *Gateway*** — the server-side deployment and operations of the Linux gateway plus the enrolled-device VPN connection profiles. It is **deliberately distinct** from the catalog's **"Microsoft Tunnel for MAM — Mobile Access to Intranet Resources"** products, which are **MAM-app-client** scoped (unenrolled/BYOD app-level tunnel). If you need app-level access on unmanaged devices, use that product; if you need to **build and run the gateway** for enrolled devices, this is the one. The two are complementary — cross-referenced, not duplicated.

What you get

• A plain-language **Solution Overview** — how the Tunnel Gateway works, the split-tunnel vs full-tunnel decision, the client-platform support reality (iOS/iPadOS + Android are the natively supported Tunnel clients; Windows/macOS caveats stated honestly), and the standard deployment model.

• A **Requirements Guide** with verify / implement-if-missing for every solution-specific prerequisite (Linux host, container runtime, TLS certificate, outbound endpoints, client app), beyond the category-wide Intune prerequisites.

• An **Implementation Guide** of exact, ordered steps — Site, Server Configuration, host install, TLS, per-platform connection profiles, Conditional Access — every object named and reused.

• Five **automation scripts**: three read-only PowerShell (Microsoft Graph) for discovery, validation, and fleet/health reporting, plus two Linux shell scripts for host prerequisite checking and gateway health + log collection. A KQL query is included for monitoring.

• **Testing** and a **Troubleshooting + Operations Runbook** for ongoing support (server health, cert expiry, connectivity, common failures).

What this delivers

A **production Microsoft Tunnel Gateway** — installed on a supported Linux host, TLS-secured, configured with the right routing model, fronting a Site that can scale to multiple servers behind a load balancer — giving managed devices secure VPN access to internal resources, with connection profiles deployed by platform, Conditional Access applied, monitoring in place, and a repeatable ops runbook for the team that keeps it running.

The deployment model

Assess → Configure → Pilot → Validate → Expand.

Outcome

Managed devices reach internal resources through a Tunnel Gateway you **own and can operate** — highly available, monitored, and supported — proven on a pilot group before you scale to the fleet.

1. Solution Overview — Microsoft Tunnel Gateway: Deployment, VPN Configuration & Ongoing Operations

What this solves

Intune-managed devices frequently need to reach resources that never leave the corporate network — an internal line-of-business web app, a file server, an internal API, an on-premises service. Microsoft Tunnel provides that path: a VPN gateway, managed through Intune, that terminates client VPN connections and forwards traffic to your internal network. The catch is that the **Gateway itself** is not a cloud service you toggle on — it is a **container that runs on a Linux server you own**. Someone has to install it, give it a trusted TLS certificate, configure how it routes traffic, deploy the client VPN profiles, make it survive a single-server failure, and keep it healthy for the long run.

This solution does all of that, end to end, and then hands you the runbook to operate it.

How Microsoft Tunnel works (the parts you must understand)

Four moving pieces make up a working Tunnel deployment:

1. **The Tunnel Gateway (server side).** A Linux host runs a **container** (via Podman or Docker) that

is the Microsoft Tunnel agent. It listens on a TLS port for incoming client connections, decrypts them, and forwards the traffic to your internal network according to the routing rules you set. The host is *yours* — an on-prem VM, a VM in your datacenter, or an IaaS VM — and it needs a network path to the internal resources you want to reach.

1. **Intune configuration objects (control plane).** In the Intune admin center you create a **Site**

(the logical grouping that a public FQDN points at) and a **Server Configuration** (the routing and network settings — listener port, DNS servers, the client IP address pool, and the split/full-tunnel routes). The Server Configuration is assigned to the Site; when you install a Tunnel server it joins that Site and inherits the configuration.

1. **The client VPN connection profiles.** Intune pushes a **VPN profile** to enrolled devices telling

them how to connect to the Tunnel (the Site FQDN, the connection type = Microsoft Tunnel, the authentication method, and per-app / on-demand options). The device-side VPN client is delivered by the **Microsoft Defender for Endpoint** app on the supported mobile platforms.

1. **Identity + Conditional Access.** Users authenticate to the Tunnel through Microsoft Entra ID, and

you can gate access with a **Conditional Access** policy (for example, require a compliant device before the Tunnel connection is allowed).

A device connects → authenticates to Entra ID → the Tunnel Gateway accepts the TLS session → traffic that matches the configured routes is forwarded to the internal network → everything else either goes direct (split tunnel) or also through the Tunnel (full tunnel), per your Server Configuration.

Client-platform support — stated honestly

The brief for this solution covers connection profiles for **Windows, iOS/iPadOS, Android, and macOS**, so this package documents all four. Be aware of the real support matrix, because it decides what will actually connect:

- **iOS/iPadOS and Android are the natively supported Microsoft Tunnel client platforms.** The device

VPN client is provided by the **Microsoft Defender for Endpoint** app, and Intune ships a native **Microsoft Tunnel** VPN connection-profile type for these platforms.

- **Windows and macOS:** Microsoft's *device-tunnel* client support has historically centered on the

two mobile platforms above. Native Microsoft Tunnel VPN client support for Windows/macOS is **not guaranteed** and has differed by scenario and over time. **Verify current Microsoft guidance for your tenant** before promising Windows/macOS Tunnel connectivity — the platform matrix and connection-type availability change. This package gives you the Windows/macOS connection-profile *procedure and the exact thing to verify*, and is explicit where support is conditional, rather than inventing a capability. If Windows/macOS clients are a hard requirement and native Tunnel support is unavailable, the "where to go next" section names the adjacent approaches.

> ***Do not confuse this with the MAM Tunnel.*** *The catalog's *"Microsoft Tunnel for MAM"* products are the **app-level** tunnel for **unenrolled** devices (mobile application management). **This** solution is the **device tunnel** for **enrolled** devices and the **gateway build + operations** behind it. Same gateway technology family, different client scope. Use the MAM product for BYOD app access; use this one to stand up and run the gateway.*

Split tunnel vs full tunnel — the routing decision

Your **Server Configuration** decides how much traffic goes through the Tunnel:

- **Split tunnel (recommended default):** only traffic destined for the **include routes** (your

internal CIDR ranges) goes through the Tunnel; everything else (internet, cloud SaaS) goes direct. Lower load on the gateway, better user experience, smaller blast radius. You define **include routes** for the internal ranges and optionally **exclude routes** to carve out subnets that should *not* go through the Tunnel.

• **Full tunnel:** all device traffic is routed through the Tunnel. Use only when a policy genuinely

requires all traffic to egress through the corporate network; it puts far more load on the gateway and makes capacity planning and monitoring more important.

The routing decision, the client IP address pool, the DNS servers, and the listener port all live in the Server Configuration and are covered step-by-step in the Implementation Guide.

Scope — what this solution covers

• Linux host prerequisites and readiness checking for the Tunnel Gateway.

• Container runtime (Podman/Docker) and the Microsoft Tunnel agent install.

• TLS server certificate preparation and placement.

• Intune **Site** + **Server Configuration** (listener, DNS, client address pool, split/full-tunnel

routes).

• **Per-platform VPN connection profiles** (iOS/iPadOS, Android, plus Windows/macOS with the verified

caveats), per-app VPN and on-demand options, and the Defender client app.

• **Conditional Access** integration (require compliant device before Tunnel).

• **High availability / scale-out** (multiple servers per Site behind a load balancer) and an

upgrade strategy.

• **Monitoring, health checks, and an operations & troubleshooting runbook** for ongoing support.

Out of scope (deferred to the customer or another product)

• **The MAM / app-level tunnel for unenrolled devices** — use the catalog's *Microsoft Tunnel for MAM*

products.

• **Building your internal network, firewall, and DNS** — this solution consumes an existing internal

network and tells you exactly which paths and ports the gateway needs; your network team owns the firewall/routing changes.

• **Your PKI / certificate authority** — you supply a valid TLS server certificate and (if you choose

certificate-based client auth) a working SCEP/PKCS pipeline; standing up a CA is a separate effort.

• **Regulatory, data-residency, and acceptable-use determinations** — the guide provides the technical

controls; classification and policy decisions belong to your security/compliance functions.

• **Load-balancer product selection and licensing** — the guide states the L4/TCP passthrough

requirement; the specific appliance/service is yours to choose.

Authentication and connections used

• **Microsoft Entra ID** — user authentication to the Tunnel and the identity backbone.

• **Microsoft Intune** — Site, Server Configuration, VPN connection profiles, app deployment, Conditional Access.

• **Microsoft Graph (PowerShell)** — the read-only discovery/validation/reporting scripts read Tunnel configuration and device state via documented `deviceManagement` endpoints. Some Tunnel resources are on the Graph **beta** endpoint; the scripts note this where used — *verify current Graph coverage*.

• **The Linux host root/sudo context** — the Tunnel install and the host-side shell scripts run on the gateway server; the install registers the server with Intune using an interactive Intune-admin sign-in.

The standard deployment model

> ***Assess*** *the host, network, certificate, and licensing →* ***Configure*** *the Site, Server Configuration, install the gateway, and build the connection profiles →* ***Pilot*** *on a small device group with split tunnel and report-only Conditional Access →* ***Validate*** *connectivity, routing, health, and CA →* ***Expand*** *to the fleet, add HA servers, and enforce Conditional Access. Pattern:* ***Assess*** *→* ***Configure*** *→* ***Pilot*** *→* ***Validate*** *→* ***Expand.***

2. Requirements Guide — Microsoft Tunnel Gateway: Deployment, VPN Configuration & Ongoing Operations

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (Entra tenant, Intune subscription, admin roles, device enrollment, pilot group, network access to Microsoft cloud endpoints) are in `global-requirements-intune-solutions.md` — confirm those too; they are not repeated here.

> ***Naming convention used throughout this package:*** `<Purpose> – <Scope> – <Stage>`*. The objects you create are named exactly: Site* `Tunnel Site – HQ – Prod`*; Server Configuration* `Tunnel Server Config – Split Tunnel – Prod`*; connection profiles* `Tunnel VPN – iOS – Pilot`, `Tunnel VPN – Android – Pilot`, `Tunnel VPN – Windows – Pilot`, `Tunnel VPN – macOS – Pilot`*; Conditional Access policy* `Tunnel Access – Compliant Devices – Report-Only`*; device groups* `Tunnel Devices – Pilot` *and* `Tunnel Devices – Broad`*. Reuse these verbatim in every step, test, and script. Use* `contoso.com` / `tunnel.contoso.com` *as placeholder names — never a real tenant/FQDN.*
>
> ***"Verify current Microsoft guidance" flag.*** *Microsoft periodically changes the supported Linux distributions, container runtimes, server sizing, port requirements, and client-platform matrix for Microsoft Tunnel. Where an item below could drift, it is marked* ***[verify current Microsoft guidance]****. Confirm the current values on Microsoft Learn for your region/cloud before you buy hardware or open firewall rules.*

2.1 Intune licensing for Microsoft Tunnel

Description: An Intune plan that includes Microsoft Tunnel, assigned to the users who will connect. **Why:** Tunnel is an Intune feature; users and the deployment require appropriate Intune licensing. **Verify:** M365 admin center → **Billing** → **Licenses**; confirm Intune Plan 1 (or a bundle that includes it) is assigned to

pilot users. **[verify current Microsoft guidance]** on any Tunnel-specific licensing. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

2.2 A supported Linux host for the Gateway

Description: A dedicated Linux server (physical or VM) to run the Tunnel Gateway container. Plan one host per Tunnel server; plan **two or more** if you want high availability from day one. **Why:** The Tunnel Gateway runs as a container on Linux — this host *is* the gateway. **Verify:** You have a Linux VM/host with a supported distribution and version, adequate CPU/RAM/disk, and network access to your internal resources. Run `TUNGW-HostPrereqCheck.sh` on the host. **Implement:** Provision a Linux VM on a **supported distribution and version** with the documented minimum CPU / RAM / disk. **[verify current Microsoft guidance]** — supported distros (historically Red Hat / CentOS-family and Ubuntu), versions, and sizing change; confirm before provisioning.

2.3 Container runtime (Podman or Docker)

Description: A supported container runtime installed on the Linux host — **Podman** or **Docker**, matched to your distribution. **Why:** The Microsoft Tunnel agent runs as a container; the install script deploys it into this runtime. **Verify:** `podman --version` **or** `docker --version` returns a supported version; `TUNGW-HostPrereqCheck.sh` reports the runtime present. **Implement:** Install the runtime that matches your distro (**[verify current Microsoft guidance]** on which runtime/version pairs with which distro), enable and start its service, and confirm a test container runs.

2.4 TLS server certificate for the Gateway

Description: A TLS **server** certificate (with private key) whose subject / Subject Alternative Name matches the **public FQDN** that clients will use to reach the Tunnel (e.g. `tunnel.contoso.com`), plus the full trust chain. **Why:** Clients establish a TLS session to the gateway; the certificate must be valid, trusted by the clients, and cover the exact FQDN, or connections fail. **Verify:** You hold the certificate and private key in the format the installer expects (commonly PEM `.pem`/`.crt` + key, plus chain) and the FQDN is present in the SAN. Confirm expiry is comfortably in the future. `TUNGW-HostPrereqCheck.sh` checks for the cert files and reports the not-after date. **Implement:** Obtain a certificate from a public or internal CA for the Tunnel FQDN. **[verify current Microsoft guidance]** on the exact file format/layout the current installer expects, and place the files where setup will read them. Ensure clients trust the issuing chain (public CA = already trusted; internal CA = deploy the root via a **Trusted certificate** profile).

2.5 Public FQDN + DNS + network path

Description: A public DNS name (`tunnel.contoso.com`) that resolves to the gateway's reachable address (or to the load-balancer VIP in an HA design), and a network path from clients → gateway → internal resources. **Why:** Clients connect by FQDN; the gateway must reach the internal ranges you intend to publish. **Verify:** `nslookup tunnel.contoso.com` resolves to the intended address; from the host you can reach a representative internal resource. `TUNGW-HostPrereqCheck.sh` runs the connectivity checks. **Implement:** Create the DNS record; open the inbound **listener port** (default TCP **443**, configurable — **[verify current Microsoft guidance]**) from clients to the gateway/VIP; ensure the host can route to your internal CIDR ranges.

2.6 Outbound connectivity from the Gateway to Microsoft services

Description: Outbound HTTPS from the Linux host to the Microsoft endpoints the Tunnel agent and Intune enrollment require (Entra login, Intune/Graph management, the Tunnel service, and the container image source at install time). **Why:** The server registers with Intune, pulls its configuration, reports health, and downloads the container image — all outbound to Microsoft. **Verify:** From the host, reach `https://login.microsoftonline.com` and the Intune/Graph endpoints; `TUNGW-HostPrereqCheck.sh` probes the key endpoints. **Implement:** Allow the host outbound to Microsoft's published Intune/Entra/Tunnel URL & IP ranges and the container registry used at install. **[verify current Microsoft guidance]** for the current required FQDN/IP list.

2.7 Microsoft Defender for Endpoint app (mobile Tunnel client)

Description: The **Microsoft Defender for Endpoint** app, deployed via Intune to iOS/iPadOS and Android devices — it provides the device-side Tunnel VPN client. **Why:** On the natively supported mobile platforms, Defender for Endpoint is the client that establishes the Tunnel connection defined by the VPN profile. **Verify:** Intune → **Apps** → **All apps**; the Defender app is added and assigned to `Tunnel Devices – Pilot`. **[verify current Microsoft guidance]** on the current client app for each platform. **Implement:** Add and assign the Defender app for iOS and Android to the Tunnel device group before you assign the VPN profiles.

2.8 Client authentication method decided

Description: How devices authenticate to the Tunnel — Microsoft Entra ID (username/password + MFA) or **certificate-based** authentication (a SCEP/PKCS-issued client certificate). **Why:** The VPN connection profile references the chosen method; certificate auth additionally requires a working certificate pipeline and a trusted root on the device. **Verify:** You have decided the method; for certificate auth, a SCEP/PKCS profile and trusted-root profile exist (or are planned). **Implement:** For Entra auth, no extra pipeline. For certificate auth, stand up SCEP/PKCS (a separate effort) and plan `Tunnel Client Cert – SCEP – Pilot` and `Trusted Root – Org CA – Mobile` profiles.

2.9 Load balancer for high availability (if deploying HA)

Description: A **Layer-4 / TCP passthrough** load balancer in front of two or more Tunnel servers in the same Site, presenting the Site FQDN VIP. **Why:** HA and scale-out put multiple servers behind one FQDN; the TLS session must pass through un-terminated (L4), because the gateway terminates TLS itself. **Verify:** The load balancer does **TCP/L4 passthrough** (not TLS termination) on the listener port and health-probes the servers. **Implement:** Configure an L4/TCP passthrough VIP for the listener port across the server pool. **[verify current Microsoft guidance]** on the supported balancing model and health-probe approach.

2.10 Microsoft Graph PowerShell (for the reporting scripts)

Description: The Microsoft Graph PowerShell SDK on the admin workstation. **Why:** `TUNGW-Discovery.ps1`, `TUNGW-Validate.ps1`, and `TUNGW-Report.ps1` read Tunnel configuration and device state via Graph. **Verify:** `Get-Module Microsoft.Graph -ListAvailable` returns a version. **Implement:** `Install-Module Microsoft.Graph -Scope CurrentUser`. Consent to the read scopes (`DeviceManagementConfiguration.Read.All`, `DeviceManagementManagedDevices.Read.All`) on first connect.

2.11 Pilot and broad device groups

Description: Entra security groups `Tunnel Devices – Pilot` (a few representative devices/users) and `Tunnel Devices – Broad` (the rest). **Why:** Every deployment is piloted before fleet rollout. **Verify:** Entra → **Groups**; both groups exist. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

Checklist

- Intune licensing for Tunnel assigned to pilot users
- Supported Linux host provisioned — `TUNGW-HostPrereqCheck.sh` **[verify distro/version/sizing]**
- Container runtime (Podman/Docker) installed and running — `TUNGW-HostPrereqCheck.sh`
- TLS server certificate (+ key + chain) for the Tunnel FQDN, in the expected format
- Public FQDN + DNS + inbound listener port + internal routing path
- Outbound connectivity to Microsoft endpoints — `TUNGW-HostPrereqCheck.sh`
- Microsoft Defender for Endpoint app deployed to iOS/Android pilot devices
- Client authentication method chosen (Entra or certificate)
- Load balancer (L4/TCP passthrough) if deploying HA
- Microsoft Graph PowerShell installed + read scopes consented
- `Tunnel Devices – Pilot` and `Tunnel Devices – Broad` groups created

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation Guide — Microsoft Tunnel Gateway: Deployment, VPN Configuration & Ongoing Operations

Follow these steps **in order**. Control-plane steps are in the **Microsoft Intune admin center** (`https://intune.microsoft.com`); host steps run on the **Linux gateway server** as root/sudo. Build the control-plane configuration **first**, then install the server so it joins a Site that already has a Server Configuration waiting for it.

> ***Model:*** *Groups → Site → Server Configuration → assign config to Site → install the Linux gateway (TLS cert + register) → deploy the Defender client app → per-platform VPN connection profiles → report-only Conditional Access → Pilot gate → Expand (add HA servers, enforce CA). Pattern:* ***Assess → Configure → Pilot → Validate → Expand.***
>
> ***Portal paths and object names drift.*** *Menu labels and some settings change over time and by cloud (Commercial vs Government). Where a value could change it is flagged* ***[verify current Microsoft***

***guidance]**. Confirm the exact wording in your tenant as you go.*

Step 0 — Assess (run the prerequisite check first)

1. On the Linux host, copy and run `TUNGW-HostPrereqCheck.sh` (read-only). It reports the distribution,

kernel, container runtime, IP-forwarding readiness, the TLS cert files + expiry, inbound listener port, and outbound reachability to Microsoft endpoints and to a sample internal resource.

1. From the admin workstation, run `TUNGW-Discovery.ps1` (read-only) to inventory any existing Tunnel

Sites, Server Configurations, and VPN profiles so you do not collide with them.

1. Resolve every failed check before continuing (see the Requirements Guide).

Step 1 — Create the device groups

1. Entra → **Groups** → **New group** → **Security** → name `Tunnel Devices – Pilot` (a few representative

iOS/Android devices or their users). **Create**.

1. Create `Tunnel Devices – Broad` (the rest of the target fleet). **Create**.

Step 2 — Create the Site

The Site is the logical unit clients point at by FQDN; servers join a Site.

1. Intune → **Tenant administration** → **Microsoft Tunnel Gateway** → **Create site** (or **Sites** → **Create**).

[verify current Microsoft guidance] on the exact menu label.

1. **Name:** `Tunnel Site – HQ – Prod`.
2. **Public IP address or FQDN:** `tunnel.contoso.com` — the name clients will connect to (or the

load-balancer VIP FQDN for HA).

1. Leave **Server configuration** unset for now (you assign it in Step 3b) or select it after Step 3a.

Save / Create.

Step 3a — Create the Server Configuration

This holds the routing and network settings every server in the Site inherits.

1. Intune → **Tenant administration** → **Microsoft Tunnel Gateway** → **Server configurations** → **Create**.
2. **Name:** `Tunnel Server Config – Split Tunnel – Prod`.
3. **Connection type / routing:** choose **Split tunnel** (recommended). Configure:

• **IP address range (client address pool):** the private CIDR the gateway hands to connected

clients (must not overlap your internal ranges), e.g. `169.254.0.0/16` or a reserved private range you own. **[verify current Microsoft guidance]** on defaults/limits.

- **Server port (listener):** the inbound TLS port, default **443** (change only if 443 is taken;

it must match the firewall rule and the client profile). **[verify current Microsoft guidance]**.

- **DNS servers:** the internal DNS server IPs clients should use while connected.
- **DNS suffix search** (optional): your internal domain suffix, e.g. `corp.contoso.com`.
- **Include routes (split tunnel):** the internal CIDR ranges that SHOULD go through the Tunnel,

e.g. `10.0.0.0/8`, `172.16.0.0/12`. Only these are tunneled; everything else goes direct.

- **Exclude routes** (optional): subnets inside the include ranges that should NOT be tunneled.
- *(Full tunnel alternative:* set the configuration to route **all** traffic through the Tunnel —

use only if policy requires all egress via corporate; expect higher gateway load.)*

1. **(Optional) Automatic server upgrades:** if you want servers to self-update, enable the automatic

upgrade option and set the maintenance window. **[verify current Microsoft guidance]** — leave off for a controlled first rollout; enable later for fleet operations.

1. **Save / Create.**

Step 3b — Assign the Server Configuration to the Site

1. Open `Tunnel Site – HQ – Prod` → set **Server configuration** = `Tunnel Server Config – Split Tunnel

– Prod`. **Save.** The Site now has routing waiting for the first server.

Step 4 — Install the Tunnel Gateway on the Linux host

Run on the Linux server as root/sudo. **[verify current Microsoft guidance]** for the current download URL, script name, and cert file layout — these change.

1. **Place the TLS certificate files** where the installer expects them (commonly a setup directory

for the full-chain certificate `.pem`/`.crt` and the private key). Confirm ownership/permissions are restrictive.

1. **Download and run the Microsoft Tunnel install script** (historically retrieved from the Microsoft

Tunnel download URL, e.g. `mstunnel-setup`), as root. The script pulls the container image into your Podman/Docker runtime and steps through setup.

1. **Provide the requested inputs** during setup: the TLS certificate + key + chain, the Site the

server should join (`Tunnel Site – HQ – Prod`), and the listener port if not default.

1. **Register the server with Intune:** when prompted, sign in with an account that has the required

Intune role (Intune Administrator or the delegated Tunnel role) to enroll the server into the Site.

1. **Confirm the service is running:** the host runs a Tunnel service (e.g. a `systemd` unit) and the

management CLI (`mst-cli`). Run `mst-cli server show` / the health command to confirm the server is up and connected. Run `TUNGW-HealthCheck.sh` for a consolidated status.

1. In Intune → **Microsoft Tunnel Gateway** → **Health status / Servers**, confirm the new server appears

under `Tunnel Site – HQ – Prod` and reports **healthy**.

Step 5 — Deploy the Microsoft Defender for Endpoint client app (mobile)

1. Intune → **Apps** → **All apps** → **Add** → add **Microsoft Defender for Endpoint** for **iOS/iPadOS** and for **Android**. **[verify current Microsoft guidance]** on the current Tunnel client app.

1. **Assign** each to `Tunnel Devices – Pilot`. Wait until pilot devices report the app installed — the VPN profile needs the client present to connect.

Step 6 — Create the per-platform VPN connection profiles

Create one profile per platform, all pointing at `Tunnel Site – HQ – Prod`. Assign each to `Tunnel Devices – Pilot` first.

Step 6a — iOS/iPadOS (natively supported)

1. Intune → **Devices** → **Configuration** → **Create** → **New policy**.
2. **Platform:** iOS/iPadOS. **Profile type: Templates** → **VPN**. **Create**.
3. **Name:** `Tunnel VPN – iOS – Pilot`.
4. **Connection type: Microsoft Tunnel**. **[verify current Microsoft guidance]** on the label.
5. **Base VPN:** select the Tunnel **Site** = `Tunnel Site – HQ – Prod`; set the **connection name** users see; set **authentication** = the method chosen in Requirements (Entra, or the client certificate `Tunnel Client Cert – SCEP – Pilot`).

1. **(Optional) Per-app VPN / On-demand:** to scope the Tunnel to specific apps, configure **per-app VPN **and associate the target apps; set** on-demand rules** if the Tunnel should connect automatically for internal domains.

1. **Assignments** → **Add groups** → `Tunnel Devices – Pilot`. **Next** → **Create.**

Step 6b — Android (natively supported)

1. **Devices** → **Configuration** → **Create** → **New policy** → **Platform:** Android Enterprise → **Profile type: VPN**. Create**.

1. **Name:** `Tunnel VPN – Android – Pilot`.
2. **Connection type: Microsoft Tunnel**; **Base VPN** → **Site** = `Tunnel Site – HQ – Prod`; set the connection name and authentication method.

1. **(Optional) Per-app VPN:** associate the apps that should trigger the Tunnel.
2. **Assignments** → `Tunnel Devices – Pilot`. **Next** → **Create.**

Step 6c — Windows (verify support first)

> ***Before creating this profile, confirm Windows is currently a supported Microsoft Tunnel client*** *for your tenant/cloud.* ***[verify current Microsoft guidance]*** *— native device-Tunnel client support has centered on iOS/Android; do not promise Windows Tunnel connectivity until verified.*

1. If (and only if) supported: **Devices** → **Configuration** → **Create** → **New policy** → **Platform:**

Windows 10 and later → **Profile type: Templates** → **VPN** (or the current Tunnel-capable VPN type). **Create**.

1. **Name:** `Tunnel VPN – Windows – Pilot`. Select the **Microsoft Tunnel** connection type / Site `Tunnel Site – HQ – Prod` if offered, set authentication, and assign to `Tunnel Devices – Pilot`.

1. If native Tunnel support is **not** available for Windows, do **not** fabricate a profile — see

Best Practices → "Where to go next" for the adjacent approach, and record the gap.

Step 6d — macOS (verify support first)

> ***Same caveat as Windows. [verify current Microsoft guidance]*** *on macOS device-Tunnel client support before creating this profile.*

1. If supported: **Devices** → **Configuration** → **Create** → **New policy** → **Platform:** macOS → **Profile

type: VPN **(Tunnel-capable type)** → Name:** `Tunnel VPN – macOS – Pilot`; Site `Tunnel Site – HQ – Prod`; authentication; assign to `Tunnel Devices – Pilot`.

1. If unsupported, record the gap and use the adjacent approach in Best Practices — do not invent a

capability.

Step 7 — Conditional Access (report-only first)

Gate the Tunnel so only compliant devices connect — but start in **report-only** so you never lock out your pilot.

1. Entra → **Security** → **Conditional Access** → **Policies** → **New policy**.
2. **Name:** `Tunnel Access – Compliant Devices – Report-Only`.
3. **Users:** the pilot users/group. **Target resources:** the app(s) the Tunnel fronts, per your

design. **Grant: Require device to be marked as compliant** (and/or require MFA per policy).

1. **Enable policy: Report-only. Create.**
2. Review report-only results (Entra → **Monitoring** → **Sign-in logs** → **Report-only** tab) before you

move it to **On** at Expand.

Step 8 — Apply on a pilot device and validate

1. On a pilot iOS/Android device with the Defender app installed, force an Intune sync and open the

Defender/Tunnel client; initiate the VPN connection defined by `Tunnel VPN – iOS – Pilot` / `Tunnel VPN – Android – Pilot`.

1. Confirm the VPN connects, DNS resolves internal names, and you can reach a resource inside an

include route (and, for split tunnel, that non-included traffic goes direct).

1. Run `TUNGW-Validate.ps1` for the device and `TUNGW-HealthCheck.sh` on the server. Complete the

Testing Guide.

Step 9 — Pilot gate

Do not expand until, on `Tunnel Devices – Pilot`: the VPN profile shows **Succeeded**; the client connects; an internal include-route resource is reachable; split-tunnel routing behaves as configured; the server reports **healthy**; and report-only Conditional Access shows the expected result with no unexpected blocks. **Rollback:** unassign the VPN profiles from `Tunnel Devices – Pilot` (devices drop the profile at next sync); the gateway keeps running with no client impact.

Step 10 — Expand to production

1. Add `Tunnel Devices – Broad` to each validated VPN profile (or clone to `– Broad` names).
2. **Add HA servers:** install one or more additional Tunnel servers into `Tunnel Site – HQ – Prod`

(repeat Step 4 on each host), place them behind the **L4/TCP passthrough** load balancer, and point the Site FQDN at the VIP. Validate each server reports healthy with `TUNGW-HealthCheck.sh`.

1. **Move Conditional Access to On:** after report-only looks clean, switch

`Tunnel Access – Compliant Devices – Report-Only` to **On** (consider renaming to `– Enforced`).

1. Run `TUNGW-Report.ps1` for fleet profile + server-health status.

Order of operations

`Assess (prereq check + discovery)` → `Groups` → `Site` → `Server Configuration` → `assign config to Site` → `install gateway (TLS + register)` → `Defender app` → `VPN profiles (iOS, Android, +Windows/macOS if supported)` → `report-only CA` → `Pilot gate` → `Expand (HA servers + LB + enforce CA)`

5. Testing Guide — Microsoft Tunnel Gateway: Deployment, VPN Configuration & Ongoing Operations

Run after the pilot profiles apply (Implementation Step 8) and again before expanding. Use `TUNGW-Discovery.ps1` (config inventory), `TUNGW-Validate.ps1` (per-device profile/config), `TUNGW-Report.ps1` (fleet + server-health rollup), `TUNGW-HostPrereqCheck.sh` (host readiness), and `TUNGW-HealthCheck.sh` (gateway health + logs).

Test 1 — Gateway server is healthy in Intune

Do: Intune → **Tenant administration** → **Microsoft Tunnel Gateway** → **Health status / Servers**; confirm the server(s) under `Tunnel Site – HQ – Prod`. Also run `TUNGW-HealthCheck.sh` on the host.

Expected: Server(s) report **healthy** with a recent check-in; the container/service is running; CPU, memory, latency, and throughput are within normal range.

Test 2 — TLS certificate valid and matches the FQDN

Do: `TUNGW-HostPrereqCheck.sh` reports the certificate not-after date; from a client network, confirm the TLS certificate presented for `tunnel.contoso.com` is trusted and covers the FQDN. **Expected:** Certificate is trusted, the FQDN is in the SAN, and expiry is comfortably in the future.

Test 3 — VPN connection profile deployed

Do: Intune → **Devices** → **Configuration** → open `Tunnel VPN – iOS – Pilot` / `Tunnel VPN – Android – Pilot` → **Device status**; or run `TUNGW-Validate.ps1` for a pilot device. **Expected:** Pilot devices show **Succeeded** (not Error/Pending); the profile targets `Tunnel Devices – Pilot`.

Test 4 — Client connects to the Tunnel

Do: On a pilot device with the Defender client installed, initiate the Tunnel VPN connection. **Expected:** The VPN establishes and shows connected; no authentication or certificate errors.

Test 5 — Internal resource reachable through an include route

Do: With the Tunnel connected, browse/ping an internal resource inside a configured **include route** (e.g. an internal web app on `10.x.x.x`), and confirm internal DNS resolves its name. **Expected:** The internal resource is reachable and its name resolves via the configured DNS servers.

Test 6 — Split-tunnel routing behaves as configured

Do: With the Tunnel connected, check that traffic to a **non-included** destination (e.g. a public site) egresses **directly**, not through the Tunnel (compare public IP with/without the Tunnel, or trace the route). **Expected:** Only include-route traffic goes through the gateway; everything else goes direct (split tunnel). For a full-tunnel config, all traffic egresses via the corporate network instead.

Test 7 — Conditional Access (report-only) evaluates as intended

Do: Entra → **Monitoring** → **Sign-in logs** → **Report-only** tab; find the pilot user's Tunnel-related sign-in and inspect the `Tunnel Access – Compliant Devices – Report-Only` result. **Expected:** The policy would **grant** for a compliant device and **block** a non-compliant one — no unexpected blocks before you switch it to On.

Test 8 — High availability failover (if HA deployed)

Do: With two+ servers behind the L4 load balancer, take one server out of rotation (or stop its service) and reconnect a client; run `TUNGW-HealthCheck.sh` on the remaining server. **Expected:** Clients still connect via the surviving server through the VIP; the healthy server reports up; no client-visible outage beyond a reconnect.

Test 9 — Fleet + health report

Do: Run `TUNGW-Report.ps1`. **Expected:** A clear per-device VPN-profile deployment status across `Tunnel Devices – Pilot` / `– Broad`, plus a server-health rollup for the Site.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–7 pass on `Tunnel Devices – Pilot` (Test 8 if HA is in the pilot)
Broad pass	Tests 1–9 pass after assigning `Tunnel Devices – Broad`, adding HA servers, and enforcing Conditional Access

6. Troubleshooting Guide + Operations Runbook — Microsoft Tunnel Gateway

Use `TUNGW-Discovery.ps1` (config inventory), `TUNGW-Validate.ps1` (per-device profile/config), `TUNGW-Report.ps1` (fleet + server-health rollup), `TUNGW-HostPrereqCheck.sh` (host readiness), and `TUNGW-HealthCheck.sh` (gateway health + `--collect` log bundle). Portal labels, `mst-cli` subcommands, ports, and endpoints change — treat **[verify current Microsoft guidance]** items as re-checkable.

The core scenario — "the client won't connect to the Tunnel"

Work this tree top to bottom; the cause is almost always in the first four rungs.

1. **Is the gateway server healthy?**

• Intune → **Microsoft Tunnel Gateway** → **Health status**; run `TUNGW-HealthCheck.sh` on the host. If

the container/service is down or the server is unhealthy, no client connects. See section A.

1. **Is the TLS certificate valid, trusted, and matching the FQDN?**

• An expired, untrusted, or wrong-FQDN certificate is the most common "it worked yesterday" cause.

`TUNGW-HealthCheck.sh` prints days-remaining; confirm the client trusts the chain and the FQDN is in the SAN. See section B.

1. **Is the client app installed and is a Tunnel VPN profile assigned + applied?**

• The mobile client is the **Microsoft Defender for Endpoint** app — confirm it is installed. Then

confirm the VPN profile (`Tunnel VPN – iOS – Pilot` / `Tunnel VPN – Android – Pilot`) is assigned to the device's group and shows **Succeeded** (`TUNGW-Validate.ps1`). See section C.

1. **Can the client reach the FQDN + listener port, and can the gateway reach internal resources?**

• DNS must resolve `tunnel.contoso.com` to the gateway/VIP; the inbound listener port (default 443)

must be open end to end; the gateway host must route to the internal include ranges. See section D.

1. **Is authentication succeeding (Entra or certificate)?**

• Check Entra sign-in logs for the user; for certificate auth confirm the client cert deployed and the trusted root is present. See section E.

1. **Is Conditional Access blocking?**

• If a CA policy (`Tunnel Access – Compliant Devices – Report-Only`) moved to **On**, a non-compliant device is blocked by design. Check the Entra sign-in **Conditional Access** result. See section F.

A. Gateway server unhealthy / container down

Symptom	Cause	Fix
Server shows unhealthy in Intune	Container/service stopped, or lost outbound to Microsoft	`TUNGW-HealthCheck.sh`; confirm the `mstunnel` service and container are running and the host reaches `login.microsoftonline.com` / `graph.microsoft.com` / `manage.microsoft.com`. Restart the service only per your change process
Server never appears after install	Registration/enrolment didn't complete	Re-run the install/registration step signed in with the required Intune role; confirm the server joined `Tunnel Site – HQ – Prod`
Health degrades under load	Under-sized host / full tunnel saturating the gateway	Check CPU/memory/throughput in Intune health; add an HA server (Step 10) or switch to split tunnel; re-verify sizing **[verify current Microsoft guidance]**

B. TLS certificate problems

• **Expired:** `TUNGW-HealthCheck.sh` reports EXPIRED — rotate the certificate on the host (historically `mst-cli tls_cert` update — **[verify current subcommand]**) and confirm clients reconnect.

• **Untrusted chain:** clients fail TLS because they don't trust the issuer. For an internal CA, deploy the root via an Intune **Trusted certificate** profile to the device group. Public-CA certs are already trusted.

• **FQDN mismatch:** the SAN must contain the exact name clients use (`tunnel.contoso.com`). Reissue the certificate with the correct SAN if it does not.

• **Cert files in the wrong place/format:** the installer expects a specific file layout/format —

[verify current Microsoft guidance] and re-place the full-chain + key.

C. Client app missing or VPN profile not applied

• **Defender app not installed:** the VPN profile has nothing to drive. Confirm the **Microsoft Defender for Endpoint** app is assigned and installed on the device before troubleshooting the profile.

• **Profile Error/Pending:** confirm the device is enrolled and in `Tunnel Devices – Pilot/– Broad`, the profile platform matches the device, and force a sync. `TUNGW-Validate.ps1` shows the per-device apply state and the Site/routing it resolves to.

• **Windows/macOS profile "does nothing":** confirm native Microsoft Tunnel client support currently exists for that platform in your cloud — **[verify current Microsoft guidance]**. If it does not, the profile cannot connect; use the adjacent approach in Best Practices rather than forcing it.

D. Networking — FQDN, port, or internal routing

Symptom	Cause	Fix
Client can't reach the gateway at all	DNS or inbound port	`nslookup tunnel.contoso.com` resolves to the gateway/VIP; confirm the listener port (default 443) is open from client networks to the gateway/VIP end to end
Connects but no internal resource works	Include routes / gateway routing	Confirm the internal CIDRs are in the Server Configuration **include routes**, and the gateway host can actually route to them (`TUNGW-HostPrereqCheck.sh --internal <ip>`)
Internal names don't resolve	DNS servers not set	Set the internal **DNS servers** (and suffix) in the Server Configuration
Client-pool collision	Client address pool overlaps internal range	Change the client IP address range so it does not overlap any include route
HA: some connections fail	Load balancer terminating TLS or bad health probe	The LB must be **L4/TCP passthrough** (never terminate TLS); fix the health probe so only healthy servers receive traffic

E. Authentication failures

• **Entra auth:** check Entra → **Sign-in logs** for the user; resolve MFA/licensing/user-state issues there. Confirm the user has the Tunnel/Intune license.

• **Certificate auth:** confirm the client certificate (`Tunnel Client Cert – SCEP – Pilot`) deployed

and the **trusted root** (`Trusted Root – Org CA – Mobile`) is present on the device; a missing chain or an un-issued client cert fails auth silently. Validate via the SCEP/PKCS solution's checks.

F. Conditional Access blocking

- If `Tunnel Access – Compliant Devices – Report-Only` was switched to **On**, a **non-compliant**

device is blocked by design. Confirm in the Entra sign-in **Conditional Access** result.

- Before enforcing, always review the **report-only** results (Test 7) so you don't lock out working

devices. Roll back to report-only if enforcement caused unexpected blocks, remediate compliance, then re-enforce.

G. Diagnostics quick reference

```
# Tenant-side inventory (Sites, Server Configs, servers/health, Tunnel VPN profiles) — read-only
.\TUNGW-Discovery.ps1
# Per-device VPN profile state + resolved Site/routing — read-only
.\TUNGW-Validate.ps1 -DeviceName 'iPad-Pilot-01'
# Fleet profile status + server-health rollup + KQL — read-only
.\TUNGW-Report.ps1
```

```
# Host readiness before install — read-only
./TUNGW-HostPrereqCheck.sh --cert /etc/mstunnel/private/site.pem --port 443 --internal 10.10.0.10 --fqdn t
# Running-gateway health + optional log bundle — read-only
./TUNGW-HealthCheck.sh --collect
```

In Intune, check **Tenant administration** → **Microsoft Tunnel Gateway** → **Health status** (server health) and **Devices** → **Configuration** → **(VPN profile)** → **Device status** (per-device result).

Operations Runbook (ongoing support)

Run this cadence to keep a production Tunnel Gateway healthy. Everything here is read-only except the explicitly gated maintenance actions, which follow your change process.

Daily

- **Server health:** `TUNGW-Report.ps1` (server-health rollup) or Intune health status — every server

in each Site should be **healthy** with a recent check-in.

- **Profile deployment:** scan `TUNGW-Report.ps1` for any spike in **Error** on the Tunnel VPN profiles.

Weekly

- **Certificate expiry:** `TUNGW-HealthCheck.sh` on each server — act on any WARN (default: <30 days).
- **Capacity:** review CPU/memory/throughput/connection counts in Intune health; if trending high, plan

an HA server or move from full to split tunnel.

- **Conditional Access:** confirm the enforced Tunnel CA policy result looks normal (no surprise blocks).

Monthly / as needed

- **Certificate rotation:** rotate well before expiry (historically `mst-cli tls_cert` update — **[verify current subcommand]**); validate clients reconnect (Tests 2, 4, 5).
- **Server upgrades (rolling):** upgrade **one server at a time** so the Site stays up; after each, confirm it returns healthy (`TUNGW-HealthCheck.sh`). If automatic upgrades are enabled, verify the maintenance window and post-upgrade health.
- **Log collection for escalation:** `TUNGW-HealthCheck.sh --collect` produces a log tarball to attach to a Microsoft support case; review it before sharing (it may contain host/network detail).
- **Re-verify drift items:** re-check the **[verify current Microsoft guidance]** items (supported distro/version, runtime, ports, endpoint list, client-platform matrix) before each new rollout wave.

When to escalate

If the server is healthy, the certificate is valid and trusted, DNS + listener port + internal routing are confirmed, the client app is installed, the VPN profile shows **Succeeded**, and authentication + Conditional Access are passing — but clients still cannot use internal resources — collect a log bundle (`TUNGW-HealthCheck.sh --collect`) and the `TUNGW-Validate.ps1` output and engage **Microsoft support** with the gateway/agent version and the exact symptom.

7. Best Practices — Microsoft Tunnel Gateway: Deployment, VPN Configuration & Ongoing Operations

Keep-it-clean guidance

- **Configure the control plane before you install the host.** Create the Site and Server Configuration first so the server joins a Site that already has routing waiting for it — cleaner than installing a server with nowhere to land.
- **Default to split tunnel.** Route only the internal include-route CIDRs through the Tunnel; let internet and SaaS traffic go direct. It lowers gateway load, improves user experience, and shrinks the blast radius. Reserve full tunnel for cases where policy genuinely requires all egress via corporate.
- **Do not overlap the client address pool with internal ranges.** The IP address range the gateway hands to clients must not collide with any include-route subnet, or routing breaks in confusing ways.
- **Treat the TLS certificate as a first-class operational asset.** Track its expiry, rotate it well before it lapses, and make sure clients trust the issuing chain (deploy the internal root via a Trusted certificate profile if you used an internal CA). An expired or untrusted cert is the single most common "everything suddenly broke" cause — monitor it (Test 2, and the health check script).
- **Deploy the Defender client app before the VPN profile.** The connection profile has nothing to

drive if the client app is not yet installed on the device.

• **Pilot with report-only Conditional Access.** Never enforce "require compliant device" on the Tunnel before you have watched report-only results — a misconfigured CA policy locks out the very devices you are testing. Move to On only after report-only is clean.

• **Build high availability in deliberately.** Put two or more servers in one Site behind an **L4/TCP passthrough** load balancer (the gateway terminates TLS itself — never terminate TLS at the LB), point the Site FQDN at the VIP, and validate failover (Test 8) before you rely on it.

• **Upgrade with a rolling strategy.** Upgrade one server at a time so the Site stays up; if you enable automatic upgrades, set a maintenance window and confirm each server returns healthy afterward.

• **Monitor continuously, not reactively.** Watch server health (CPU, memory, latency, throughput, connection count) and certificate expiry in Intune and via `TUNGW-HealthCheck.sh`; the ops runbook in the Troubleshooting Guide turns these signals into actions.

• **Keep the naming convention.** `<Purpose> – <Scope> – <Stage>` across the Site, Server Configuration, profiles, CA policy, and groups keeps the deployment auditable and repeatable across waves.

• **Respect the version-drift flags.** Distros, container runtimes, ports, sizing, and the client-platform matrix change — re-verify the **[verify current Microsoft guidance]** items before each new rollout rather than trusting a stale value.

Where to go next (adjacent catalog areas)

• **Unenrolled / BYOD app access?** Use the catalog's **"Microsoft Tunnel for MAM — Mobile Access to Intranet Resources"** (and its GCC High variant) — the app-level tunnel for unmanaged devices. This gateway solution is device-tunnel for enrolled devices; the MAM product covers the BYOD app case.

• **Certificate-based client authentication?** Pair this with the Intune **SCEP/PKCS certificate** solutions to issue client certificates and deploy the trusted root, then set the VPN profile to certificate auth.

• **Windows/macOS internal access where native Tunnel client support is unavailable?** Use the platform's supported native VPN / Always On VPN approach or an alternative access method for those endpoints, and revisit the **[verify current Microsoft guidance]** matrix periodically — do not force an unsupported Tunnel client.

• **Tightening device access?** Combine with the **Conditional Access**, **Device Compliance**, and **Defender for Endpoint** solutions to make "compliant + healthy device" the gate for the Tunnel.

• **Zero-touch onboarding of the client fleet?** Pair with the **Autopilot / enrollment** solutions so new devices arrive enrolled, Defender-equipped, and Tunnel-ready.

8. Deliverables Summary — Microsoft Tunnel Gateway: Deployment, VPN Configuration & Ongoing Operations

Documentation

Document	Purpose
Executive Summary	Business case, what's in the box, and the device-tunnel-vs-MAM scope boundary
1. Solution Overview	How the Tunnel Gateway works; split vs full tunnel; honest client-platform support; deployment model
2. Requirements Guide	Every solution-specific prerequisite with Verify / Implement-if-missing (host, runtime, TLS cert, endpoints, client app)
Global Requirements (Intune Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps — Site, Server Configuration, host install + TLS, per-platform VPN profiles, Conditional Access
5. Testing Guide	Validation procedures with pass/fail gates (connectivity, routing, health, CA, HA failover)
6. Troubleshooting Guide + Operations Runbook	Symptom → cause → fix for common failures, plus ongoing-support runbook (health, cert expiry, upgrades, log collection)
7. Best Practices	Field-proven recommendations and where to go next

Scripts

Script	What it does
`TUNGW-Discovery.ps1`	PowerShell (Microsoft Graph), **read-only** — inventories Tunnel Sites, Server Configurations, servers/health, and VPN connection profiles in the tenant
`TUNGW-Validate.ps1`	PowerShell, **read-only** — per-device: confirms the Tunnel VPN profile is assigned and applied, resolves the Site/Server Configuration it points at, and reports the effective routing/auth settings
`TUNGW-Report.ps1`	PowerShell, **read-only** — fleet-wide VPN-profile deployment status plus a per-server health rollup for a Site; includes a KQL monitoring query for connection/health telemetry
`TUNGW-HostPrereqCheck.sh`	Linux shell, **read-only** — gateway host readiness: distribution/kernel, container runtime, IP-forwarding, TLS cert files + expiry, inbound listener port, and outbound reachability to Microsoft + internal resources

Script	What it does
`TUNGW-HealthCheck.sh`	Linux shell, **read-only** — gateway health + **log collection**: container/service status via `mst-cli`, TLS cert expiry, connectivity, and a support-bundle/log tarball for troubleshooting

What the buyer receives

- This solution as a **Word** document and **PDF**.
- All five **scripts** (three PowerShell + two Linux shell) in a single ZIP.
- A repeatable process: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

Outcome

A production Microsoft Tunnel Gateway — installed on a supported Linux host, TLS-secured, configured with the right routing model, highly available, monitored, and supported by a repeatable ops runbook — giving managed devices secure VPN access to internal resources, proven on a pilot before scaling to the fleet.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

TUNGW-Discovery.ps1

Purpose: Establishes the "what exists today" baseline before you deploy or change a Tunnel Gateway: 1. Lists Microsoft Tunnel Sites (the FQDN a Site fronts). 2. Lists Tunnel Server Configurations (routing type, listener port, DNS, client address pool, split-tunnel include/exclude routes). 3. Lists Tunnel servers and their reported health/last check-in per Site. 4. Lists VPN configuration profiles whose connection type is Microsoft Tunnel, per platform. Use it so you do not collide with existing Sites/configs and to confirm objects after changes. HOW TO RUN Install the Graph SDK once: Install-Module Microsoft.Graph -Scope CurrentUser Then: powershell -ExecutionPolicy Bypass -File .\TUNGW-Discovery.ps1 Sign in when prompted (read scopes only). EXPECTED RESULTS A printed inventory of Tunnel Sites, Server Configurations, servers + health, and Tunnel VPN profiles. No changes are made. .NOTES White-label. Read-only. Uses documented Microsoft Graph deviceManagement endpoints via Invoke-MgGraphRequest. Several Microsoft Tunnel resources are exposed on the Graph BETA endpoint (microsoftTunnelSite / microsoftTunnelConfiguration / microsoftTunnelServer) — clearly noted where used. Verify current Graph coverage for your tenant/cloud (Commercial vs Government differ). > [CmdletBinding()] param()

How to run: Install the Graph SDK once: Install-Module Microsoft.Graph -Scope CurrentUser Then: powershell -ExecutionPolicy Bypass -File .\TUNGW-Discovery.ps1 Sign in when prompted (read scopes only). EXPECTED RESULTS A printed inventory of Tunnel Sites, Server Configurations, servers + health, and Tunnel VPN profiles. No changes are made. .NOTES White-label. Read-only. Uses documented Microsoft Graph deviceManagement endpoints via Invoke-MgGraphRequest. Several Microsoft Tunnel resources are exposed on the Graph BETA endpoint (microsoftTunnelSite / microsoftTunnelConfiguration / microsoftTunnelServer) — clearly noted where used. Verify current Graph coverage for your tenant/cloud (Commercial vs Government differ). > [CmdletBinding()] param()

TUNGW-HealthCheck.sh

Purpose: The ongoing-operations health probe for a running Tunnel Gateway server: 1. Reports the Tunnel service + container status (via mst-cli and the container runtime). 2. Reports the TLS certificate expiry (days remaining) so you rotate before it lapses. 3. Probes outbound reachability to Microsoft endpoints (management/health path). 4. Optionally COLLECTS a support-bundle/log tarball for troubleshooting or escalation. It only reads state and (with --collect) copies logs into a tarball — it does not change, restart, or reconfigure the gateway. HOW TO RUN chmod +x TUNGW-HealthCheck.sh ./TUNGW-HealthCheck.sh # health summary only ./TUNGW-HealthCheck.sh --cert /etc/mstunnel/private/site.pem ./TUNGW-HealthCheck.sh --collect # also gather a log tarball into /tmp EXPECTED RESULTS A health summary (service/container up, cert days-remaining, endpoint reachability). With --collect, the path to a logs tarball you can attach to a support case. .NOTES White-label. Read-only (health read + optional log copy). mst-cli subcommands, service names, container names, and endpoints change — VERIFY CURRENT MICROSOFT GUIDANCE. Run as root/sudo so mst-cli and logs are readable.

How to run: chmod +x TUNGW-HealthCheck.sh ./TUNGW-HealthCheck.sh # health summary only ./TUNGW-HealthCheck.sh --cert /etc/mstunnel/private/site.pem ./TUNGW-HealthCheck.sh --collect # also gather a log tarball into /tmp EXPECTED RESULTS A health summary (service/container up, cert

days-remaining, endpoint reachability). With --collect, the path to a logs tarball you can attach to a support case. .NOTES White-label. Read-only (health read + optional log copy). mst-cli subcommands, service names, container names, and endpoints change — VERIFY CURRENT MICROSOFT GUIDANCE. Run as root/sudo so mst-cli and logs are readable.

TUNGW-HostPrereqCheck.sh

Purpose: Verifies a Linux host is ready to run the Microsoft Tunnel Gateway BEFORE you install: 1. Distribution + kernel version (so you can compare against current supported list). 2. Container runtime present (Podman or Docker) and its service state. 3. IP forwarding readiness (the gateway routes client traffic to internal networks). 4. TLS certificate files present + not-after (expiry) date. 5. Inbound listener port free/reachable and outbound reachability to Microsoft endpoints and to a sample internal resource. It ONLY reads/probes — it changes nothing on the host. HOW TO RUN chmod +x TUNGW-HostPrereqCheck.sh ./TUNGW-HostPrereqCheck.sh \ --cert /etc/mstunnel/certs/site.pem \ --port 443 \ --internal 10.10.0.10 \ --fqdn tunnel.contoso.com All flags optional; sensible defaults are used and clearly reported. EXPECTED RESULTS A PASS/WARN/FAIL line per check. Resolve every FAIL (and review WARN) before running the Microsoft Tunnel installer. .NOTES White-label. Read-only (no changes). Distribution/runtime/port/endpoint specifics change — VERIFY CURRENT MICROSOFT GUIDANCE for the supported distro/version, runtime, ports, and the required outbound Microsoft endpoint list before provisioning.

How to run: chmod +x TUNGW-HostPrereqCheck.sh ./TUNGW-HostPrereqCheck.sh \ --cert /etc/mstunnel/certs/site.pem \ --port 443 \ --internal 10.10.0.10 \ --fqdn tunnel.contoso.com All flags optional; sensible defaults are used and clearly reported. EXPECTED RESULTS A PASS/WARN/FAIL line per check. Resolve every FAIL (and review WARN) before running the Microsoft Tunnel installer. .NOTES White-label. Read-only (no changes). Distribution/runtime/port/endpoint specifics change — VERIFY CURRENT MICROSOFT GUIDANCE for the supported distro/version, runtime, ports, and the required outbound Microsoft endpoint list before provisioning.

TUNGW-Report.ps1

Purpose: The operations "one page" for the Tunnel deployment: 1. For each Tunnel VPN profile, rolls up per-device apply state (Succeeded / Error / Pending) across all assigned devices. 2. For each Site, rolls up server health (healthy vs unhealthy, last check-in). 3. Prints a KQL query (see EXPECTED RESULTS) you can paste into a Log Analytics / monitoring workspace for Tunnel connection + health telemetry, if you route Intune/Tunnel signals there. Run it at the Pilot gate, before Expand, and on a schedule for ongoing operations. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TUNGW-Report.ps1 Optional: -CsvPath .\tungw-report.csv (writes the per-device rows to CSV) Sign in when prompted (read scopes only). EXPECTED RESULTS A summary table per VPN profile (Succeeded/Error/Pending counts), a server-health rollup per Site, and a ready-to-paste KQL monitoring query. No changes are made. .NOTES White-label. Read-only. Documented Microsoft Graph deviceManagement endpoints; Tunnel Site/server resources on Graph BETA (noted). Verify current Graph coverage for your cloud. > [CmdletBinding()] param([string] $CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\TUNGW-Report.ps1 Optional: -CsvPath .\tungw-report.csv (writes the per-device rows to CSV) Sign in when prompted (read scopes only). EXPECTED RESULTS A summary table per VPN profile (Succeeded/Error/Pending counts), a server-health rollup per Site, and a ready-to-paste KQL monitoring query. No changes are made. .NOTES White-label. Read-only. Documented Microsoft Graph deviceManagement endpoints; Tunnel Site/server

resources on Graph BETA (noted). Verify current Graph coverage for your cloud. > [CmdletBinding()] param([string] $CsvPath)

TUNGW-Validate.ps1

Purpose: Answers "is this device correctly set up for the Tunnel?" without changing anything: 1. Resolves the managed device by name and confirms it is enrolled and (optionally) compliant. 2. Lists the Tunnel VPN configuration profile(s) targeting the device's groups and the per-device apply state (Succeeded / Error / Pending). 3. Resolves the Site + Server Configuration the profile points at and prints the effective routing (listener port, DNS, client address pool, include/exclude routes) and auth method. Use it during the Pilot gate (Implementation Step 8/9) and whenever a single device won't connect. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TUNGW-Validate.ps1 -DeviceName 'iPad-Pilot-01' Sign in when prompted (read scopes only). EXPECTED RESULTS A per-device report: enrollment/compliance, the assigned Tunnel VPN profile(s) + apply state, and the resolved Site/Server Configuration routing + auth. No changes are made. .NOTES White-label. Read-only. Documented Microsoft Graph deviceManagement endpoints; Tunnel Site/Config resources are on Graph BETA (noted). Verify current Graph coverage for your cloud. > [CmdletBinding()] param([Parameter(Mandatory)] [string] $DeviceName)

How to run: powershell -ExecutionPolicy Bypass -File .\TUNGW-Validate.ps1 -DeviceName 'iPad-Pilot-01' Sign in when prompted (read scopes only). EXPECTED RESULTS A per-device report: enrollment/compliance, the assigned Tunnel VPN profile(s) + apply state, and the resolved Site/Server Configuration routing + auth. No changes are made. .NOTES White-label. Read-only. Documented Microsoft Graph deviceManagement endpoints; Tunnel Site/Config resources are on Graph BETA (noted). Verify current Graph coverage for your cloud. > [CmdletBinding()] param([Parameter(Mandatory)] [string] $DeviceName)